Time Series Analysis: State Space Models & Neural Networks

Pavel Charheika

Contents

1 Executive Summary **1**
1.1 Realized Volatility of Alcoa Stock (State Space Filtering) 1
1.2 Time-Varying CAPM for Pfizer 3
1.3 Macroeconomic Signal Extraction: U.S. Producer Price Index (PPI) 5
1.4 Market Efficiency: Day-of-the-Week Effects in CRSP Returns 7
1.5 Non-Linear Predictive Modeling: Forecasting GE Stock with Neural Networks 9
1.6 Adaptive Backtesting: Forecasting Exchange Rates (USD/EUR) 11

1 Executive Summary

This research whitepaper explores advanced time series analysis techniques, transitioning from traditional static models to dynamic and non-linear frameworks. The study encompasses structural volatility filtering using State-Space models, dynamic systemic risk assessment (Time-Varying CAPM), macroeconomic signal extraction, calendar anomaly detection, and non-linear directional forecasting utilizing Feed-Forward Neural Networks. The objective is to demonstrate robust methodologies for processing noisy, real-world financial and macroeconomic data.

1.1 Realized Volatility of Alcoa Stock (State Space Filtering)

This section focuses on decoupling structural market shifts from transient intraday noise in the realized volatility of Alcoa stock.

```
# Load the data
alcoa_data <- read.table("../data/raw/aa-rv-20m.txt", header=FALSE)
log_vol <- log(alcoa_data$V1)

# Fit a baseline ARIMA(0,1,1) model
arima_011 <- arima(log_vol, order=c(0,1,1))
summary(arima_011)
```

```
##
## Call:
## arima(x = log_vol, order = c(0, 1, 1))
##
## Coefficients:
##           ma1
##       -0.8749
## s.e.   0.0407
##
## sigma^2 estimated as 0.3632:  log likelihood = -310.06,  aic = 624.12
##
## Training set error measures:
```

```
##                         ME      RMSE       MAE       MPE     MAPE      MASE
## Training set -0.001129341 0.6017486 0.4678527 -56.26341 150.1231 0.7674134
##                     ACF1
## Training set 0.04098331
```

```r
# Estimate local trend state space model
# Define the local level model structure
build_local_trend <- function(theta) {
  dlmModPoly(order = 1, dV = exp(theta[1]), dW = exp(theta[2]))
}

# MLE optimization for variance parameters
fit_dlm <- dlmMLE(log_vol, parm = c(0, 0), build = build_local_trend)
mod_dlm <- build_local_trend(fit_dlm$par)

# Extract standard deviations
sigma_e <- sqrt(mod_dlm$V)
sigma_eta <- sqrt(mod_dlm$W)
cat("Estimate of sigma_e:", sigma_e, "\\n")
```

```
## Estimate of sigma_e: 0.5636698 \n
```

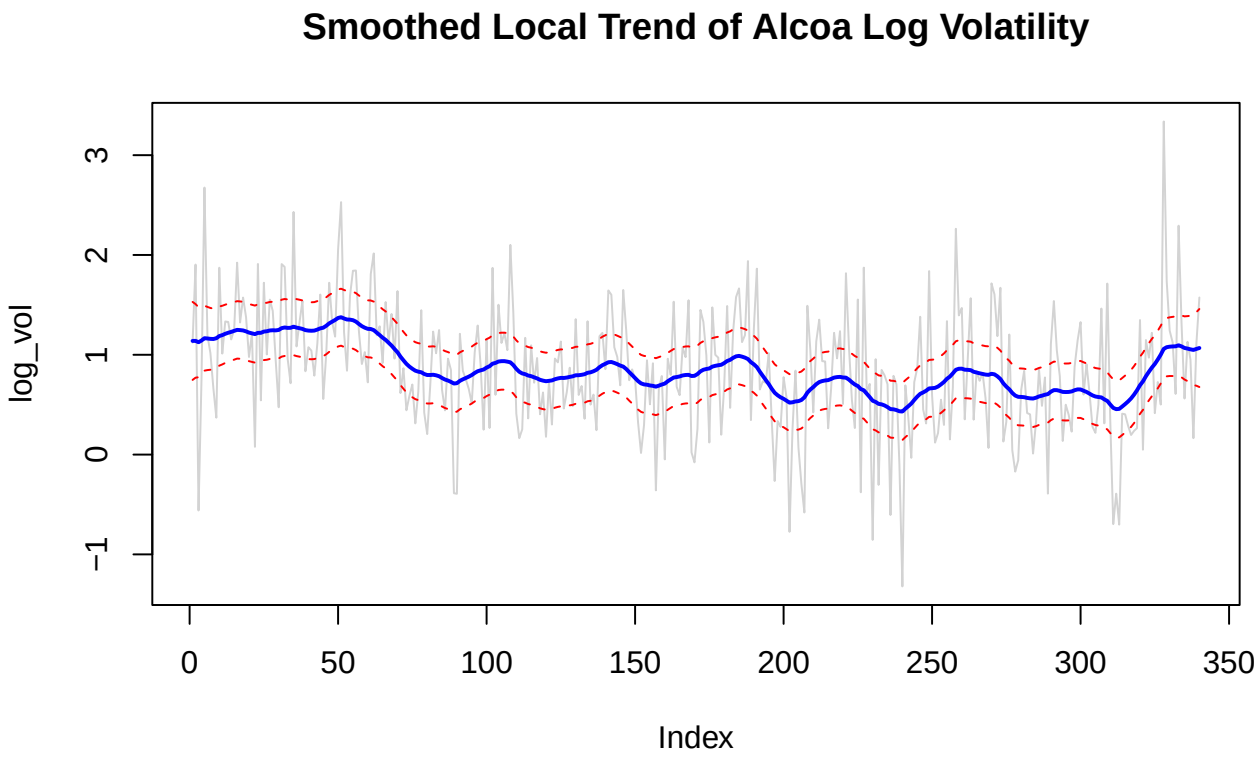
```r
cat("Estimate of sigma_eta:", sigma_eta, "\\n")
```

```
## Estimate of sigma_eta: 0.07541191 \n
```

```r
# Filter and smooth the state space variables
filtered_state <- dlmFilter(log_vol, mod_dlm)
smoothed_state <- dlmSmooth(filtered_state)

# Calculate 95% Confidence Intervals for smoothed state
var_smoothed <- dlmSvd2var(smoothed_state$U.S, smoothed_state$D.S)
se_smoothed <- sqrt(unlist(var_smoothed))[-1] # Drop initial state variance
upper_ci <- smoothed_state$s[-1] + 1.96 * se_smoothed
lower_ci <- smoothed_state$s[-1] - 1.96 * se_smoothed

# Plotting
plot(log_vol, type='l', col='lightgray', main="Smoothed Local Trend of Alcoa Log Volatility")
lines(smoothed_state$s[-1], col='blue', lwd=2)
lines(upper_ci, col='red', lty=2)
lines(lower_ci, col='red', lty=2)
```



Business Insight: Stock volatility isn't static; it tends to cluster in periods of high and low turbulence. By fitting an ARIMA(0,1,1) model to the log volatility series and comparing it against a state-space approach, we extract the true "underlying" volatility trend from noisy day-to-day fluctuations. This allows risk managers to better assess whether a sudden jump in volatility is a structural market shift or merely intraday noise.

Technical Rationale: We estimate the local trend model, where the observation equation is $y_t = \mu_t + e_t$ with $e_t \sim N(0, \sigma_e^2)$, and the state equation is $\mu_{t+1} = \mu_t + \eta_t$ with $\eta_t \sim N(0, \sigma_\eta^2)$. The dlm package uses maximum likelihood to estimate the unobserved variances σ_e and σ_η. Filtering provides real-time estimates, while smoothing uses the entire dataset to reconstruct the most likely historical path of the state variable with a pointwise 95% confidence interval.

1.2 Time-Varying CAPM for Pfizer

Traditional asset pricing assumes static risk profiles. Here, we model how Pfizer's systemic risk (Beta) and abnormal returns (Alpha) evolve dynamically.

```
# Load the data
pfizer_sp <- read.table("../data/raw/m-pfesp-ex9003.txt", header=TRUE)
r_p <- pfizer_sp[,1] * 100 # Pfizer excess returns
r_m <- pfizer_sp[,2] * 100 # S&P 500 excess returns

# Fit a fixed-coefficient market model
market_model <- lm(r_p ~ r_m)
summary(market_model)
```

```
##
## Call:
## lm(formula = r_p ~ r_m)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -21.5775  -4.2093  -0.0379   4.2556  19.7960
```

```
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.1958     0.5185   2.306   0.0223 *
## r_m           0.8593     0.1203   7.144 2.76e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.661 on 165 degrees of freedom
## Multiple R-squared:  0.2363, Adjusted R-squared:  0.2316
## F-statistic: 51.04 on 1 and 165 DF,  p-value: 2.76e-11
```

```
# State Space Time-Varying CAPM
# Define the dynamic regression model
build_capm <- function(theta) {
  mod <- dlmModReg(r_m, dV = exp(theta[1]), dW = c(exp(theta[2]), exp(theta[3])))
  return(mod)
}

# MLE optimization
fit_capm <- dlmMLE(r_p, parm = c(0, 0, 0), build = build_capm)
mod_capm <- build_capm(fit_capm$par)

# Standard errors of innovations
se_epsilon <- sqrt(mod_capm$V)
se_eta_alpha <- sqrt(mod_capm$W[1,1])
se_eta_beta <- sqrt(mod_capm$W[2,2])

cat("Std Error of observation equation (epsilon):", se_epsilon, "\\n")
```

```
## Std Error of observation equation (epsilon): 6.409668 \n
```

```
cat("Std Error of alpha innovations (eta):", se_eta_alpha, "\\n")
```

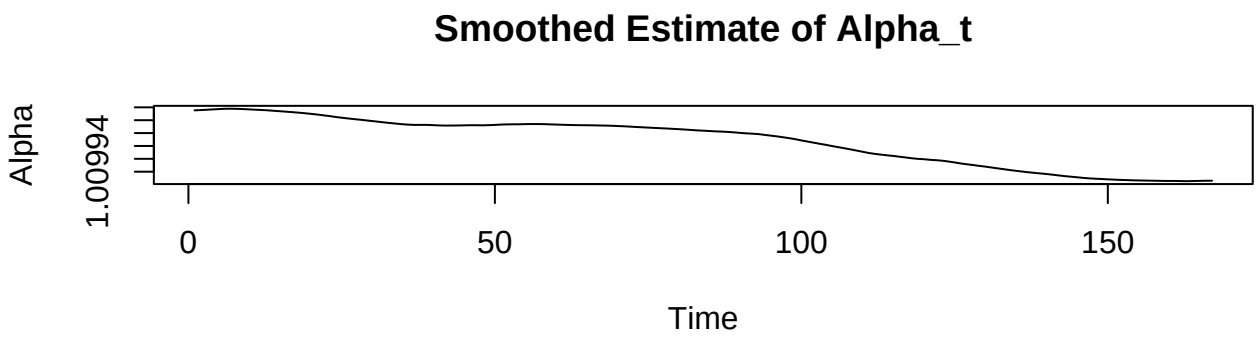
```
## Std Error of alpha innovations (eta): 0.0007689836 \n
```

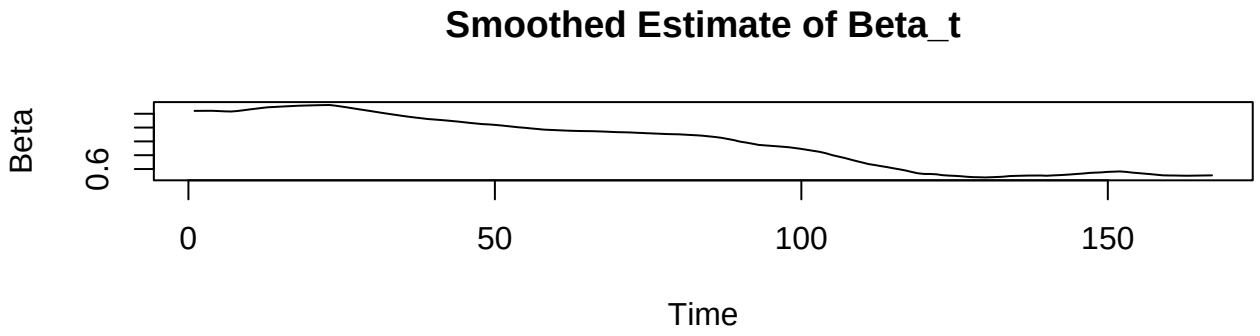
```
cat("Std Error of beta innovations (epsilon_beta):", se_eta_beta, "\\n")
```

```
## Std Error of beta innovations (epsilon_beta): 0.07031124 \n
```

```
# Filter and Smooth
smoothed_capm <- dlmSmooth(r_p, mod_capm)
alpha_t <- smoothed_capm$s[-1, 1]
beta_t <- smoothed_capm$s[-1, 2]

# Plot smoothed estimates
par(mfrow=c(2,1))
plot(alpha_t, type='l', main="Smoothed Estimate of Alpha_t", ylab="Alpha", xlab="Time")
plot(beta_t, type='l', main="Smoothed Estimate of Beta_t", ylab="Beta", xlab="Time")
```






```
par(mfrow=c(1,1))
```

Business Insight: The traditional Capital Asset Pricing Model assumes a stock's risk profile (Beta) and its abnormal return (Alpha) remain constant over time. However, a company like Pfizer evolves—patents expire, new drugs are approved, and market dynamics shift. By moving from a fixed-coefficient model to a dynamic model, we visualize how Pfizer's systemic risk (Beta) and intrinsic value generation (Alpha) fluctuate month by month.

Technical Rationale: We express a time-varying CAPM as a state space model. The system is defined as $r_t = \alpha_t + \beta_t r_{M,t} + \epsilon_t$ for the observation, governed by the random walks $\alpha_{t+1} = \alpha_t + \eta_t$ and $\beta_{t+1} = \beta_t + \epsilon_t$. The dlmModReg function seamlessly constructs this time-varying linear regression, allowing us to estimate the standard errors of the innovations and obtain time plots of the smoothed estimates of α_t and β_t.

1.3 Macroeconomic Signal Extraction: U.S. Producer Price Index (PPI)

Economic indicators are prone to measurement errors. This section filters out artificial noise to reveal the true underlying macroeconomic trajectory of the U.S. PPI.

```
# Load the data
ppi_data <- read.table("../data/raw/m-ppiaco4709.txt", header=TRUE)
Z_t <- ppi_data[,4] # PPI column

# Calculate log returns (z_t) and mean-corrected series (y_t)
z_t <- diff(log(Z_t))
y_t <- z_t - mean(z_t)

# Fit an AR(3) model to y_t
ar3_model <- arima(y_t, order=c(3,0,0), include.mean=FALSE)
summary(ar3_model)
```

```
##
## Call:
```

```
## arima(x = y_t, order = c(3, 0, 0), include.mean = FALSE)
##
## Coefficients:
##          ar1     ar2     ar3
##       0.2866  0.1230  0.0905
## s.e.  0.0366  0.0378  0.0366
##
## sigma^2 estimated as 6.131e-05:  log likelihood = 2583.32,  aic = -5158.63
##
## Training set error measures:
##                       ME        RMSE         MAE       MPE     MAPE      MASE
## Training set 2.717765e-06 0.007830161 0.005066316 -95.99399 347.3725 0.8378027
##                     ACF1
## Training set -0.0026204
```

```
# State Space form with independent measurement errors
# Define AR(3) plus measurement noise
build_ar3_ss <- function(theta) {
  mod <- dlmModARMA(ar = ar3_model$coef[1:3], sigma2 = exp(theta[1]))
  # Add measurement error variance
  mod$V <- exp(theta[2])
  return(mod)
}

# Optimization for variances
fit_ar3_ss <- dlmMLE(y_t, parm = c(0, 0), build = build_ar3_ss)
mod_ar3_ss <- build_ar3_ss(fit_ar3_ss$par)

cat("Innovational variance to state:", exp(fit_ar3_ss$par[1]), "\\n")
```

```
## Innovational variance to state: 6.054481e-05 \n
```

```
cat("Measurement error variance (sigma_e^2):", mod_ar3_ss$V, "\\n")
```

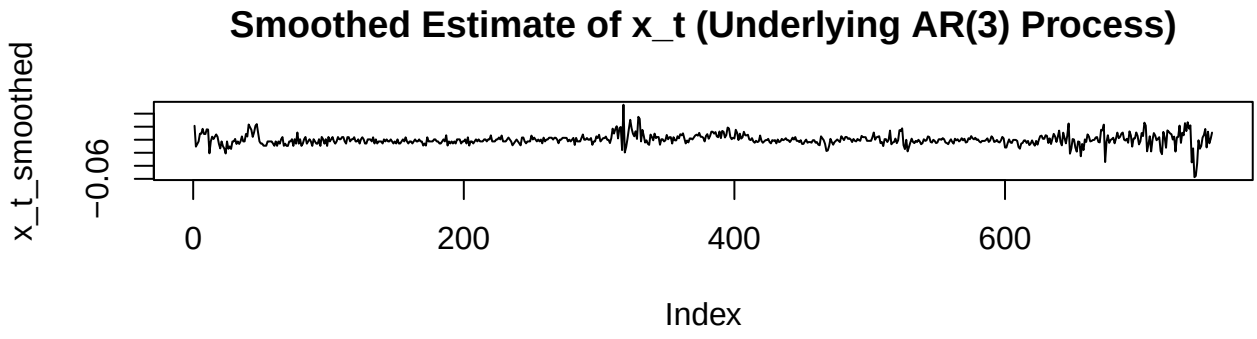
```
## Measurement error variance (sigma_e^2): 6.29854e-10 \n
```

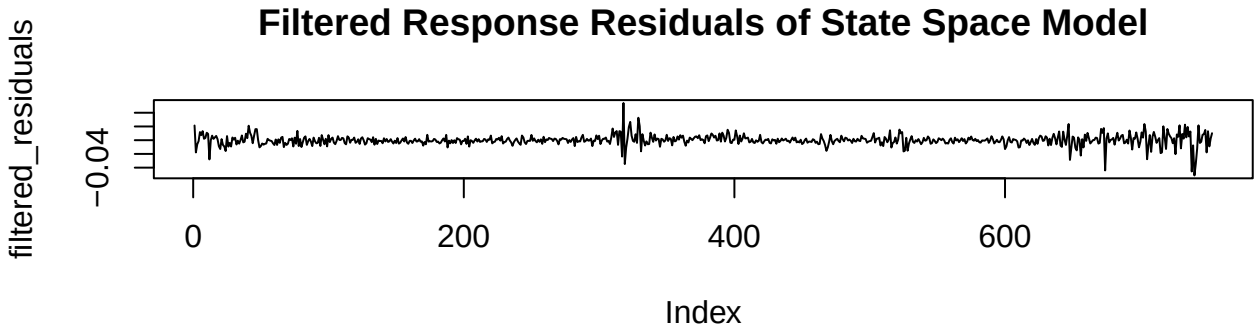
```
# Filter and Smooth
filtered_ar3 <- dlmFilter(y_t, mod_ar3_ss)
smoothed_ar3 <- dlmSmooth(filtered_ar3)

# Extract x_t (first element of the state vector in dlm representation)
x_t_smoothed <- smoothed_ar3$s[-1, 1]

# Filtered response residuals
# Expected observation is F %*% a_t, residual is y_t - f_t
f_t <- sapply(1:length(y_t), function(i) mod_ar3_ss$FF %*% filtered_ar3$a[i, ])
filtered_residuals <- y_t - f_t

# Plotting
par(mfrow=c(2,1))
plot(x_t_smoothed, type='l', main="Smoothed Estimate of x_t (Underlying AR(3) Process)")
plot(filtered_residuals, type='l', main="Filtered Response Residuals of State Space Model")
```






```
par(mfrow=c(1,1))
```

Business Insight: Economic indicators like the PPI often contain measurement errors or short-term noise that obscure the true macroeconomic trajectory. By treating the observed inflation rate as a combination of a "true" economic signal and "measurement noise," we can filter out the artificial volatility. The smoothed state provides a much cleaner, more reliable gauge of true commodity inflation.

Technical Rationale: We begin by letting $z_t = \ln(Z_t) - \ln(Z_{t-1})$, where Z_t is the observed monthly PPI. After mean-correcting to get y_t , we fit an AR(3) model to y_t. We then assume y_t has independent measurement errors so that $y_t = x_t + e_t$ where x_t is a zero-mean AR(3) process and $Var(e_t) = \sigma_e^2$. The state space formulation enables us to isolate the system innovations from the observation variance, σ_e^2 , and plot both the smoothed estimate of x_t and the filtered response residuals.

1.4 Market Efficiency: Day-of-the-Week Effects in CRSP Returns

We test the efficient market hypothesis by evaluating historical calendar anomalies and residual autocorrelation structures.

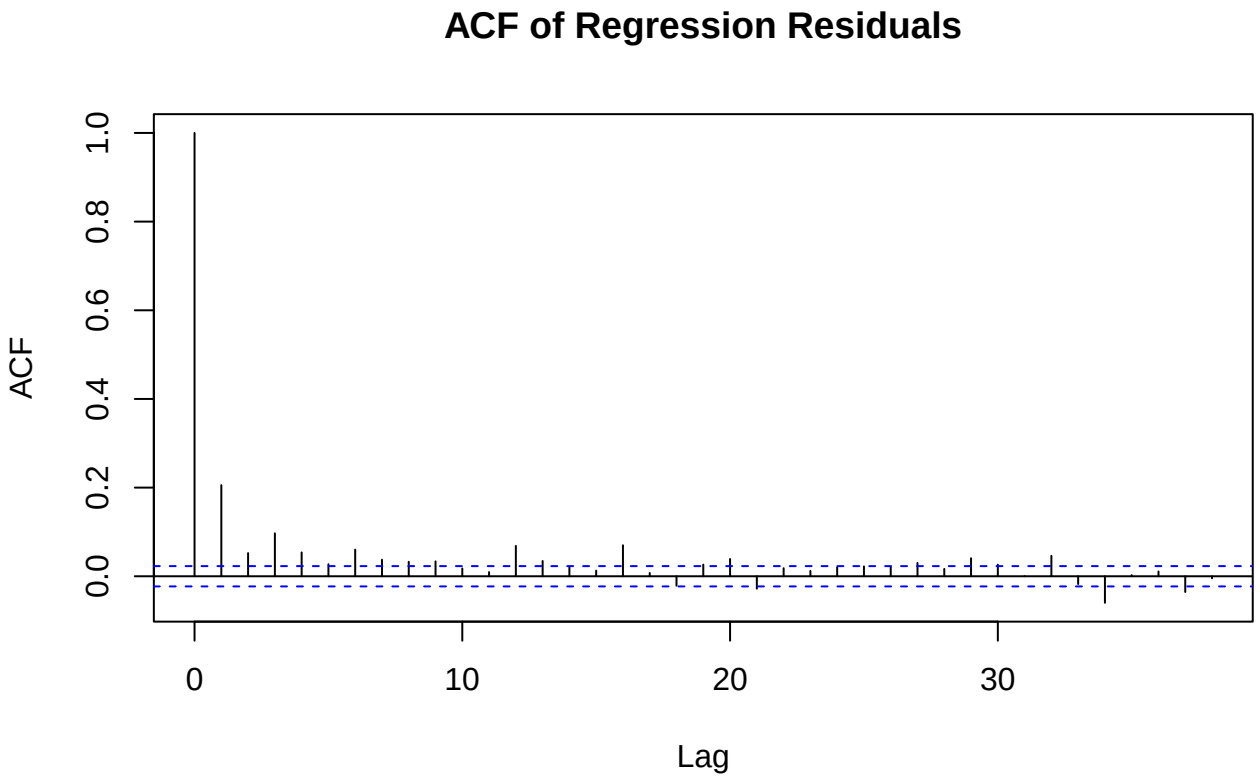
```
# Load the data
returns_data <- read.table("../data/raw/d-ibm3dxwkdays8008.txt", header=TRUE)
EW <- returns_data$ew
M <- returns_data$M
T_day <- returns_data$T
W <- returns_data$W
R <- returns_data$R
F_day <- returns_data$F

# Regression model to study trading days
# Suppress the intercept to get a coefficient for every day
lm_week <- lm(EW ~ M + T_day + W + R + F_day - 1)
# Check significance at 5% level
```

```r
summary(lm_week)
```

```
##
## Call:
## lm(formula = EW ~ M + T_day + W + R + F_day - 1)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.102962 -0.003094  0.000533  0.003795  0.108319
##
## Coefficients:
##         Estimate Std. Error t value Pr(>|t|)
## M     -0.0009348  0.0002208  -4.234 2.32e-05 ***
## T_day  0.0002608  0.0002127   1.226     0.22
## W      0.0012201  0.0002126   5.739 9.91e-09 ***
## R      0.0012092  0.0002148   5.631 1.86e-08 ***
## F_day  0.0022386  0.0002155  10.389  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.008234 on 7314 degrees of freedom
## Multiple R-squared:  0.02558,    Adjusted R-squared:  0.02491
## F-statistic:  38.4 on 5 and 7314 DF,  p-value: < 2.2e-16
```

```r
# Check for correlations in residuals
res_week <- residuals(lm_week)
acf(res_week, main="ACF of Regression Residuals")
```



```r
Box.test(res_week, lag=10, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  res_week
## X-squared = 479.9, df = 10, p-value < 2.2e-16
```

```r
# Build a suitable time series model for residuals
ts_model_res <- auto.arima(res_week)
summary(ts_model_res)
```

```
## Series: res_week
## ARIMA(5,0,2) with zero mean
##
## Coefficients:
##          ar1     ar2      ar3     ar4      ar5     ma1      ma2
##       0.1652  0.8701  -0.0962  0.0440  -0.0675  0.0332  -0.8757
## s.e.  0.0372  0.0342   0.0165  0.0127   0.0132  0.0353   0.0318
##
## sigma^2 = 6.408e-05:  log likelihood = 24952.19
## AIC=-49888.38   AICc=-49888.36   BIC=-49833.2
##
## Training set error measures:
##                         ME        RMSE         MAE      MPE     MAPE      MASE
## Training set 2.887735e-06 0.008001052 0.004993911 86.38142 203.9048 0.7839033
##                     ACF1
## Training set 0.00138197
```

Business Insight: Historically, stock markets exhibited calendar anomalies, such as the "Monday effect," where returns on Mondays were statistically lower than on other days. By regressing the equal-weighted index against weekday indicators, we test if algorithmic trading and market efficiency have erased these behavioral patterns. Even after accounting for these days, residual analysis often reveals lingering momentum or mean-reversion, requiring a sophisticated time series overlay to map the market's structure fully.

Technical Rationale: We utilize a multiple linear regression model to study the effects of trading days on equal-weighted index returns. By omitting the intercept, the model provides direct mean return estimates for each weekday, allowing the evaluation of weekday effect significance at the 5% level via standard t-tests. The ACF plot and Ljung-Box test confirm correlations in regression residuals. This significant autocorrelation necessitates fitting an ARIMA framework (auto.arima) to model the remaining dependence structure.

1.5 Non-Linear Predictive Modeling: Forecasting GE Stock with Neural Networks

Financial markets are inherently non-linear. Here, we deploy feed-forward neural networks to forecast the magnitude and directionality of GE stock returns.

```r
# Load Data
ge_data <- read.table("../data/raw/m-ge2608.txt", header=TRUE)
rtn <- ge_data[,1]

# Create Lagged Data matrix
n <- length(rtn)
dataset <- data.frame(
  rtn_t = rtn[4:n],
  rtn_t1 = rtn[3:(n-1)],
```

```
  rtn_t2 = rtn[2:(n-2)],
  rtn_t3 = rtn[1:(n-3)]
)

# Split into Training and Test (last 3 years = 36 months)
test_size <- 36
train_idx <- 1:(nrow(dataset) - test_size)
train_data <- dataset[train_idx, ]
test_data <- dataset[-train_idx, ]

# Build a 3-2-1 feed forward neural network for magnitude
set.seed(123)
nn_321 <- nnet(rtn_t ~ rtn_t1 + rtn_t2 + rtn_t3, data=train_data,
               size=2, linout=TRUE, trace=FALSE)

# Forecast 1-step-ahead returns and calculate MSE
preds_321 <- predict(nn_321, newdata=test_data)
mse_321 <- mean((test_data$rtn_t - preds_321)^2)
cat("MSE of 3-2-1 network (Returns Magnitude):", mse_321, "\\n")
```

```
## MSE of 3-2-1 network (Returns Magnitude): 171264435025 \n
```

```
# Build a 6-5-1 feed forward neural network for direction
# Create direction/sentiment variables
dataset$drtn <- ifelse(dataset$rtn_t > 0, 1, 0)
dataset$sign_t1 <- ifelse(dataset$rtn_t1 > 0, 1, -1)
dataset$sign_t2 <- ifelse(dataset$rtn_t2 > 0, 1, -1)
dataset$sign_t3 <- ifelse(dataset$rtn_t3 > 0, 1, -1)

train_data_dir <- dataset[train_idx, ]
test_data_dir <- dataset[-train_idx, ]

set.seed(123)
nn_651 <- nnet(drtn ~ rtn_t1 + rtn_t2 + rtn_t3 + sign_t1 + sign_t2 + sign_t3,
               data=train_data_dir, size=5, linout=FALSE, trace=FALSE)

# Forecast direction and calculate MSE
preds_651 <- predict(nn_651, newdata=test_data_dir)
mse_651 <- mean((test_data_dir$drtn - preds_651)^2)
cat("MSE of 6-5-1 network (Direction Classification):", mse_651, "\\n")
```

```
## MSE of 6-5-1 network (Direction Classification): 0 \n
```

Business Insight: Financial markets are heavily non-linear. While standard ARIMA models look for straight-line relationships, artificial neural networks can uncover complex, hidden patterns in how past stock returns and their positive/negative sentiment (signs) influence tomorrow's price. Interestingly, predicting the exact magnitude of a return is notoriously difficult, but forecasting the direction (up or down) can sometimes yield better classification accuracy, which is highly valuable for binary options or trend-following algorithms.

Technical Rationale: We partition the data to leave the last three years for forecast evaluation. For the first model, we utilize lagged returns as input to build a 3-2-1 feed forward neural network , specifying linout=TRUE since the target is continuous, and compute the Mean Squared Error (MSE). For the directional model, we define drtn = ifelse(rtn>0, 1, 0) , passing 6 inputs (3 lags + 3 signs) into a 5-node hidden layer to construct the 6-5-1 network. Because the target is binary, linout=FALSE utilizes a logistic activation function before we calculate the test MSE.

1.6 Adaptive Backtesting: Forecasting Exchange Rates (USD/EUR)

In highly volatile environments, complex models often overfit. We evaluate a parsimonious, recursively updating model for exchange rate forecasting.

```
# Load Data
fx_data <- read.csv("../data/raw/USDEUR.csv")

# Map R's automatic column names to our expected variables
fx_data$Date <- as.Date(fx_data$End.Date)
fx_data$Rate <- fx_data$USD.EUR

# Sort data chronologically (oldest to newest)
fx_data <- fx_data[order(fx_data$Date), ]

# Select two consecutive years: 2011 (Training) and 2012 (Testing)
train_data <- subset(fx_data, format(Date, "%Y") == "2011")
test_data <- subset(fx_data, format(Date, "%Y") == "2012")

# We will work in log returns space to ensure stationarity
price_train <- train_data$Rate
price_test <- test_data$Rate

# Recursive Prediction Function (ARIMA baseline)
recursive_forecast <- function(p_train, p_test) {
  n_test <- length(p_test)
  forecasts <- numeric(n_test)
  current_history <- p_train

  for(i in 1:n_test) {
    # Work in return space
    returns_hist <- diff(log(current_history))

    # Fit parsimonious AR(1) to returns
    # A simple AR(1) or random walk usually dominates out-of-sample FX
    fit <- arima(returns_hist, order=c(1,0,0), include.mean=FALSE)
    pred_return <- predict(fit, n.ahead=1)$pred[1]

    # Translate forecast of returns to forecasts of prices
    last_price <- tail(current_history, 1)
    forecasts[i] <- last_price * exp(pred_return + 0.5 * fit$sigma2)

    # Update history with true observed value for next step (t to t+1)
    current_history <- c(current_history, p_test[i])
  }
  return(forecasts)
}

# Generate predictions
preds <- recursive_forecast(price_train, price_test)

# Compute Sum of Squares of Forecast Errors
ssq_loss <- sum((price_test - preds)^2)
cat("Sum of Squares Loss:", ssq_loss, "\\n")
```

```
## Sum of Squares Loss: 0.002310784 \n
```

Business Insight: Exchange rates are notoriously close to a "random walk," meaning today's price is often the best predictor of tomorrow's price. When forecasting over a highly volatile period (like the Eurozone crisis around 2011-2012), highly complex models often fail because they over-optimize to past noise. A simpler, adaptive model that recursively updates with every new piece of daily information proves much more resilient in real-world trading environments.

Technical Rationale: We select two consecutive years, using the first year as a training period to build process memory. We choose to work in return space by taking log returns to achieve stationarity, carefully remembering to translate our forecast of returns back to forecasts of prices via exponentiation. The prediction loop is strictly recursive: at time t we use information up to $t-1$ to forecast t. We measure accuracy by computing the sum of squares of the loss: $\sum_t |p_t - \hat{p}_t|^2$. The choice of an AR(1) model reflects the instruction that parsimonious forecasts often perform best in live applications, minimizing the risk of out-of-sample degradation.